

Bash

☰ Tags

1. Struttura Base di uno Script Bash

- **Shebang:** `#!/bin/bash` indica il path dell'interprete Bash.
- **Commenti:** Usa `#` per aggiungere commenti e documentare il codice.
- **Esecuzione:** Gli script vanno resi eseguibili con `chmod +x script.sh` e possono essere eseguiti con `./script.sh`.

2. Gestione degli Argomenti

- **Posizionali:**
 - `$0` : Nome dello script.
 - `$1, $2, ...` : Argomenti passati allo script.
 - `$#` : Numero di argomenti.
 - `$@` e `$*` : Tutti gli argomenti.
- **Esempio:**

```
if [ $# -lt 1 ]; then
    echo "Errore: almeno un argomento è richiesto."
    exit 1
fi
```

3. Operatori e Condizioni

- **Operatori di Confronto:**
 - Numerici: `eq`, `ne`, `lt`, `le`, `gt`, `ge`.
 - Stringhe: `=`, `!=`, `<`, `>`, `z` (stringa vuota), `n` (stringa non vuota).
- **File:**

- `f` : File normale.
- `d` : Directory.
- `r`, `w`, `x` : Permessi di lettura, scrittura, esecuzione,
- `s` : non vuoto.
- **Struttura di un `if`:**

```
bash
Copy code
if [ condizione ]; then
    # Comandi
elif [ altra_condizione ]; then
    # Altri comandi
else
    # Comandi finali
fi
```

4. Cicli e Iterazioni

- **While Loop:**

```
while [ condizione ]; do
    # Comandi
done
```

- **For Loop:**

```
for i in {1..5}; do
    echo "Iterazione $i"
done
```

6. Variabili

- **Dichiarazione:** `variabile="valore"`.
- **Uso:** `$variabile`.
- **Scope:**
 - **Locali**

- **Ambiente:** Usa `export`.
- **Esempio:**

```
nome="Mario"
echo "Ciao, $nome!"
```

7. Comandi Utili

```
N=$1
for i in {0..$1}; do
    echo i
    $* $(( N - 1 ))
done
echo $N
```

- **Gestione Processi:**

- `&` → in background,
- `wait` → aspetta tutti i sottoprocessi in background,
- `kill` → chiude un processo.

- **Redirezioni:**

- stdout: `>` (sovrascrive stdout), `>>` (aggiunge).
- stderr: `2>` (sovrascrive), `2>>` (aggiunge).
- Dev/null: `>/dev/null 2>&1`.

- **Pipeline:** Usa `|` per collegare comandi.
- **Sostituzione di Comandi:** Usa `$(comando)`.

8. Debug

- **Opzioni:** Esegui con `bash -x script.sh` per tracciare i comandi.
- **Errori:** Usa `set -e` per terminare lo script al primo errore.

9. Best Practices

- Usa variabili significative.
- Documenta ogni passaggio con commenti.
- Valida sempre gli input.

- Controlla i permessi di file e directory.

10. Esercizi Pratici

1. Scrivere uno script che calcola il fattoriale di un numero.
2. Creare uno script che trova file più grandi di una certa dimensione.
3. Implementare uno script che verifica se un indirizzo IP è valido.

11. Risorse Aggiuntive

- **Manuale Bash:** Usa `man bash`.
- **Esempi:** Consulta `/usr/share/doc/` per script di esempio.

Expansion in Bash

1. Parameter Expansion

Utilizzata per manipolare il contenuto delle variabili.

- **Semplice:**

```
nome="Mario"
echo "Ciao, ${nome}!"
```

- **Default:**

- `${var:-valore}` : Usa `valore` se `var` è non definita o vuota.
- `${var:=valore}` : Assegna `valore` a `var` se è non definita o vuota.
- `${var:+valore}` : Usa `valore` solo se `var` è definita.
- `${var:?messaggio}` : Mostra `messaggio` e termina se `var` è non definita o vuota.

- **Lunghezza della variabile:**

```
var="test"
echo ${#var} # Output: 4
```

- **Sottostringa:**

```
var="hello"
echo ${var:1:3} # Output: "ell" (parte da indice 1, lunghezza 3)
```

- **Sostituzione:**

- `${var/pattern/sostituzione}` : Sostituisce la prima occorrenza.
- `${var//pattern/sostituzione}` : Sostituisce tutte le occorrenze.

2. Command Substitution

Permette di usare l'output di un comando come valore.

- Usando `$()` :

```
data=$(date)
echo "Oggi è: $data"
```

3. Arithmetic Expansion

Usata per eseguire calcoli.

- Sintassi: `$((espressione))`

```
a=5
b=$((a + 10))
echo $b # Output: 15
```

- Operatori:

- Aritmetici: `+`, `-`, `*`, `/`, `%`.
- Bitwise: `&`, `|`, `^`, `~`.
- Shift: `<<`, `>>`.

4. Brace Expansion

Genera sequenze di stringhe.

- Range:

```
echo {1..5} # Output: 1 2 3 4 5  
echo {a..e} # Output: a b c d
```

- Con passo:

```
echo {1..10..2} # Output: 1 3 5 7 9
```

- Liste:

```
echo {rosso,verde,blu} # Output: rosso verde blu
```

5. Tilde Expansion

Sostituisce la tilde `~` con il percorso della home directory.

- Home directory dell'utente:

```
echo ~ # Output: /home/utente
```

- Home directory di un altro utente:

```
echo ~utente # Output: /home/utente
```

6. Variable Expansion

Espansione di variabili con valore.

- Semplice:

```
var="Bash"  
echo $var # Output: Bash
```

- Quote Expansion (protezione da caratteri speciali):

```
var="Ciao*"
echo "$var" # Output: Ciao*
```

7. Pathname Expansion

Espande caratteri jolly nei percorsi.

- Esempi:

```
echo *.sh # Elenca tutti i file con estensione .sh
```